

Passo a passo — Segunda-feira, 17/05

TCC Fase 1 — Para P2

7 blocos sequenciais | P1 e P2 sempre trabalhando juntas

[NOVO] = tarefa adicionada na revisão do cronograma (não constava na versão original)

Visão geral do dia

#	Bloco	Responsável	Tempo
1	Configurar ambiente Python	P1 + P2	~45 min
2	Extração Ergast API 2014–2024	P1: 2014–2019 P2: 2020–2024	~1h
3	Extração FastF1 2014–2024	P1: 2014–2019 P2: 2020–2024	~1h30
4	Conectar OpenF1 API e validar acesso [NOVO]	P2 lidera, P1 apoia	~45 min
5	Mapear campos OpenF1 → Ergast [NOVO]	P1 lidera, P2 apoia	~30 min
6	Verificar completude dos dados	P1 + P2 juntas	~30 min
7	Inspeção final e commit	P1 + P2 juntas	~20 min

Bloco 1 — Configurar ambiente Python

Responsável: P1 + P2 (cada uma no próprio computador) | Tempo estimado: ~45 min

O que fazer

- Criar ambiente virtual e instalar todas as dependências

Comandos — rodar no terminal:

```
python -m venv venv
source venv/bin/activate      # Linux/Mac
# venv\Scripts\activate      # Windows

pip install pandas numpy scikit-learn xgboost lightgbm \
fastfl optuna adapt scipy matplotlib seaborn \
requests tqdm
```

- Fixar versões e salvar requirements.txt

```
pip freeze > requirements.txt
```

- Testar se tudo instalou corretamente

```
python -c "import pandas, numpy, xgboost, fastfl, optuna; print('OK')"
```

Pronto quando: o comando acima imprime 'OK' nos dois computadores e requirements.txt foi gerado

Bloco 2 — Extração Ergast API — base histórica 2014–2024

Responsável: P1 extrai 2014–2019 | P2 extrai 2020–2024 | Tempo estimado: ~1h

O que fazer

- P2 cria e roda `ergast_extract.py` para os anos 2020–2024

Script completo:

```
import requests, pandas as pd, time

BASE = "https://ergast.com/api/f1"
ANOS = range(2020, 2025) # P2 extrai 2020-2024

rows = []
for ano in ANOS:
    url = f"{BASE}/{ano}/results.json?limit=1000"
    data = requests.get(url).json()
    for raca in data["MRData"]["RaceTable"]["Races"]:
        for r in raca["Results"]:
            rows.append({
                "season": ano, "round": raca["round"],
                "circuit": raca["Circuit"]["circuitId"],
                "driver": r["Driver"]["driverId"],
                "constructor": r["Constructor"]["constructorId"],
                "grid": r["grid"], "finish": r["position"],
                "status": r["status"], "laps": r["laps"],
                "points": r["points"]
            })
    time.sleep(0.3) # respeitar rate limit

pd.DataFrame(rows).to_csv("ergast_raw_2020_2025.csv", index=False)
print(f"Extraídas {len(rows)} linhas")
```

- Após P1 terminar a parte dela, concatenar os dois CSVs

```
import pandas as pd
df = pd.concat([
    pd.read_csv("ergast_raw_2014_2020.csv"),
    pd.read_csv("ergast_raw_2020_2025.csv")
])
df.to_csv("ergast_raw_2014_2024.csv", index=False)
print(df.shape, df["season"].unique())
```

Pronto quando: `ergast_raw_2014_2024.csv` gerado com seasons 2014–2024 — confirmar com `df["season"].unique()`

Rate limit: a Ergast limita ~4 req/s. O `sleep(0.3)` já protege. Se retornar erro 429, aumentar para `sleep(1.0)`.

Bloco 3 — Extração FastF1 — qualifying e compostos de pneu

Responsável: P1 extrai 2014–2019 | P2 extrai 2020–2024 | Tempo estimado: ~1h30

O que fazer

- P2 cria e roda `fastf1_extract.py` para os anos 2020–2024

Script completo:

```
import fastf1, pandas as pd
fastf1.Cache.enable_cache("cache_fastf1")

ANOS = range(2020, 2025)    # P2 extrai 2020-2024

rows = []
for ano in ANOS:
    schedule = fastf1.get_event_schedule(ano, include_testing=False)
    for _, ev in schedule.iterrows():
        try:
            sess = fastf1.get_session(ano, ev["EventName"], "Q")
            sess.load(laps=True, telemetry=False, weather=False)
            laps = sess.laps[["Driver", "Sector1Time",
                              "Sector2Time", "Sector3Time", "Compound"]]
            laps["season"] = ano
            laps["round"] = ev["RoundNumber"]
            rows.append(laps)
        except Exception as e:
            print(f"Erro {ano} {ev['EventName']}: {e}")

pd.concat(rows).to_csv("fastf1_raw_2020_2025.csv", index=False)
print("FastF1 OK")
```

- Após P1 terminar, concatenar os dois CSVs (igual ao Bloco 2)

```
import pandas as pd
df = pd.concat([
    pd.read_csv("fastf1_raw_2014_2020.csv"),
    pd.read_csv("fastf1_raw_2020_2025.csv")
])
df.to_csv("fastf1_raw_2014_2024.csv", index=False)
print(df.shape)
```

Pronto quando: fastf1_raw_2014_2024.csv gerado. Erros de sessão individual (ex: qualifying cancelado) são normais — apenas logar e continuar.

Popular o cache do FastF1 pela primeira vez pode levar 1–2h dependendo da conexão. Deixar rodando em background enquanto avança para o Bloco 4, que não depende do FastF1.

Bloco 4 — Conectar OpenF1 API e validar acesso [NOVO] [NOVO]

Responsável: P2 lidera, P1 apoia | Tempo estimado: ~45 min

Por que isso é crítico

A OpenF1 é a única fonte de dados de 2025 e 2026. Sem ela, o walk-forward da Semana 2 não tem dado de validação e a curva de drift da Semana 3 não tem fonte. Validar o acesso hoje evita descobrir um problema na véspera da modelagem.

O que fazer

- Testar acesso à API e extrair uma corrida de 2025 como prova de conceito

Script completo:

```
import requests, json, pandas as pd

BASE = "https://api.openf1.org/v1"
```

```
# 1. Listar sessões de 2025
sessions = requests.get(f"{BASE}/sessions?year=2025").json()
df_sess = pd.DataFrame(sessions)
print(df_sess[["session_key", "session_name",
               "date_start", "circuit_short_name"]].head(10))

# 2. Pegar session_key da corrida de Melbourne
session_key = df_sess[
    (df_sess["circuit_short_name"] == "Melbourne") &
    (df_sess["session_name"] == "Race")
]["session_key"].values[0]

# 3. Extrair posições dessa corrida
positions = requests.get(
    f"{BASE}/position?session_key={session_key}").json()
df_pos = pd.DataFrame(positions)
print(df_pos.head())
print("Colunas:", df_pos.columns.tolist())

# 4. Salvar
df_pos.to_json("openf1_test_aus2025.json",
               orient="records", indent=2)
print("Arquivo salvo.")
```

Pronto quando: openf1_test_aus2025.json gerado com dados reais de posição da corrida

Se a API retornar lista vazia ou erro: verificar documentação em openf1.org — o nome do campo `circuit_short_name` pode variar. Reportar para P1 imediatamente para não travar o Bloco 5.

Bloco 5 — Mapear campos OpenF1 → schema Ergast [NOVO] [NOVO]

Responsável: P1 lidera, P2 apoia | Tempo estimado: ~30 min

Por que isso é crítico

A OpenF1 e o Ergast têm nomes de colunas diferentes. Se ninguém mapear isso hoje, P3 vai escrever sobre as três fontes na Semana 2 sem saber como elas se integram — e a integração pode quebrar durante o walk-forward.

O que fazer

- Comparar colunas dos dois datasets lado a lado

```
import pandas as pd, json

ergast = pd.read_csv("ergast_raw_2014_2024.csv")
print("Ergast colunas:", ergast.columns.tolist())

with open("openf1_test_aus2025.json") as f:
    openf1 = pd.DataFrame(json.load(f))
print("OpenF1 colunas:", openf1.columns.tolist())

# Verificar campos críticos
for c in ["driver_number", "position", "date", "session_key"]:
    print(f" {c}: {'OK' if c in openf1.columns else 'AUSENTE'}")
```

- Criar mapeamento_openf1_ergast.md com a tabela de equivalência

O arquivo deve conter uma tabela como esta (exemplo):

Campo OpenF1	Equivalente Ergast	Observação
--------------	--------------------	------------

driver_number	driver (driverId)	Precisa de lookup número → id
position	finish	OK direto
date	(calculado por round)	Usar schedule para relacionar
session_key	season + round	Chave composta
grid_position	grid	Verificar se OpenF1 tem — pode precisar do Ergast como fallback

Pronto quando: mapeamento_openf1_ergast.md criado com tabela de equivalência e estratégia de fallback definida — enviar para P3 até sábado 23/05

Bloco 6 — Verificar completude dos dados extraídos

Responsável: P1 + P2 juntas | Tempo estimado: ~30 min

O que fazer

- Inspeccionar os dois CSVs e identificar lacunas

```
import pandas as pd

ergast = pd.read_csv("ergast_raw_2014_2024.csv")
fastf1 = pd.read_csv("fastf1_raw_2014_2024.csv")

print("Ergast:", ergast.shape)
print("Seasons:", sorted(ergast["season"].unique()))
print("\nFastF1:", fastf1.shape)
print("Seasons:", sorted(fastf1["season"].unique()))

# Nulos por coluna
print("\nNulos Ergast:")
print(ergast.isnull().sum()[ergast.isnull().sum() > 0])

# Corridas no Ergast sem FastF1
e_rounds = set(ergast[["season", "round"]].apply(tuple, axis=1))
f_rounds = set(fastf1[["season", "round"]].apply(tuple, axis=1))
ausentes = e_rounds - f_rounds
print(f"\nCorridas sem FastF1: {len(ausentes)}")
for s,r in sorted(ausentes)[:10]:
    print(f"    Season {s}, Round {r}")
```

- Salvar relatório de ausências

```
with open("dados_ausentes.txt", "w") as f:
    f.write(f"Corridas sem FastF1 ({len(ausentes)} total):\n")
    for s,r in sorted(ausentes):
        f.write(f"    Season {s}, Round {r}\n")
```

Pronto quando: dados_ausentes.txt salvo. Lacunas esperadas: qualifying cancelados, alguns GPs de 2014–2015 com cobertura parcial no FastF1. Serão tratados na quarta.

Bloco 7 — Inspeção final e commit do dia

Responsável: P1 + P2 juntas | Tempo estimado: ~20 min

O que fazer

- Conferência cruzada — P2 roda o código de P1 e vice-versa
 - Pelo menos um CSV de cada deve abrir no computador da outra sem erro
 - Isso confirma que o requirements.txt está correto e o ambiente é reproduzível

```
import pandas as pd
df = pd.read_csv("ergast_raw_2014_2024.csv")
print(df.head(3))
print(df.dtypes)
```

- Commit de tudo no repositório compartilhado

```
git add requirements.txt \
    ergast_raw_2014_2024.csv \
    fastf1_raw_2014_2024.csv \
    openf1_test_aus2025.json \
    mapeamento_openf1_ergast.md \
    dados_ausentes.txt

git commit -m "feat: extração inicial Ergast + FastF1 + OpenF1 - 17/05"
git push
```

Pronto quando: todos os 6 arquivos no repositório e abrindo sem erro nos dois computadores

Entregáveis do dia — todos no repositório até o fim da segunda

Arquivo	Gerado por	Usado quando
requirements.txt	P1 + P2	Reprodução do ambiente
ergast_raw_2014_2024.csv	P1 + P2	Base de limpeza — terça
fastf1_raw_2014_2024.csv	P1 + P2	Feature engineering — qui/sex
openf1_test_aus2025.json	P2	Validação OpenF1 — S2
mapeamento_openf1_ergast.md	P1	P3 escreve metodologia — S2
dados_ausentes.txt	P1 + P2	Imputação — quarta

Pontos de atenção

- P1 e P2 trabalham juntas o tempo todo. Se uma travar em qualquer bloco, parar juntas e resolver antes de avançar.
- O FastF1 (Bloco 3) é o mais demorado. Deixar rodando em background e adiantar os Blocos 4 e 5 enquanto espera.
- Se a OpenF1 apresentar problema de acesso ou schema incompatível no Bloco 4, parar tudo e resolver — ela é dependência crítica da Semana 2.
- O mapeamento_openf1_ergast.md precisa chegar para P3 até sábado 23/05 para ela escrever a subseção de fontes na Semana 2.